

```
shubham@k8s-crio-master:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-crio-master	Ready	control-plane,master	6m3s	v1.23.6	192.168.122.21	<none>	Ubuntu 20.04.4 LTS	5.4.0-109-generic	cri-o://1.23.2
k8s-crio-worker	Ready	<none>	103s	v1.23.6	192.168.122.22	<none>	Ubuntu 20.04.4 LTS	5.4.0-109-generic	cri-o://1.23.2

```
shubham@k8s-crio-master:~$
```

Figure 6: Kubernetes nodes with CRI-O runtime

```
shubham@k8s-crio-master:~$ kubectl get pods --all-namespaces -o wide
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
kube-system	coredns-64897985d-mpnlr	1/1	Running	0	5m6s	10.244.0.2	k8s-crio-master	<none>	<none>
kube-system	coredns-64897985d-wxmsc	1/1	Running	0	5m6s	10.244.0.3	k8s-crio-master	<none>	<none>
kube-system	etcd-k8s-crio-master	1/1	Running	0	5m28s	192.168.122.21	k8s-crio-master	<none>	<none>
kube-system	kube-apiserver-k8s-crio-master	1/1	Running	0	5m24s	192.168.122.21	k8s-crio-master	<none>	<none>
kube-system	kube-controller-manager-k8s-crio-master	1/1	Running	0	5m32s	192.168.122.21	k8s-crio-master	<none>	<none>
kube-system	kube-flannel-ds-4prc6	1/1	Running	0	73s	192.168.122.22	k8s-crio-worker	<none>	<none>
kube-system	kube-flannel-ds-b64hx	1/1	Running	0	3m6s	192.168.122.21	k8s-crio-master	<none>	<none>
kube-system	kube-proxy-kv5zb	1/1	Running	0	5m7s	192.168.122.21	k8s-crio-master	<none>	<none>
kube-system	kube-proxy-vg4bt	1/1	Running	0	74s	192.168.122.22	k8s-crio-worker	<none>	<none>
kube-system	kube-scheduler-k8s-crio-master	1/1	Running	0	5m24s	192.168.122.21	k8s-crio-master	<none>	<none>

```
shubham@k8s-crio-master:~$
```

Figure 7: All the pods in the Kubernetes cluster and their status nodes with CRI-O runtime

```
shubham@k8s-docker-master:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-docker-master	Ready	control-plane,master	3m23s	v1.23.6	192.168.122.31	<none>	Ubuntu 20.04.4 LTS	5.4.0-109-generic	docker://20.10.14
k8s-docker-worker	Ready	<none>	2m7s	v1.23.6	192.168.122.32	<none>	Ubuntu 20.04.4 LTS	5.4.0-109-generic	docker://20.10.14

```
shubham@k8s-docker-master:~$
```

Figure 8: Kubernetes nodes with Docker runtime

```
shubham@k8s-docker-master:~$ kubectl get pods --all-namespaces -o wide
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
kube-system	coredns-64897985d-6v6ph	1/1	Running	0	2m31s	10.244.0.3	k8s-docker-master	<none>	<none>
kube-system	coredns-64897985d-bgrl2	1/1	Running	0	2m31s	10.244.0.2	k8s-docker-master	<none>	<none>
kube-system	etcd-k8s-docker-master	1/1	Running	0	2m44s	192.168.122.31	k8s-docker-master	<none>	<none>
kube-system	kube-apiserver-k8s-docker-master	1/1	Running	0	2m46s	192.168.122.31	k8s-docker-master	<none>	<none>
kube-system	kube-controller-manager-k8s-docker-master	1/1	Running	0	2m47s	192.168.122.31	k8s-docker-master	<none>	<none>
kube-system	kube-flannel-ds-ktcqd	1/1	Running	0	99s	192.168.122.32	k8s-docker-worker	<none>	<none>
kube-system	kube-flannel-ds-zxwj4	1/1	Running	0	116s	192.168.122.31	k8s-docker-master	<none>	<none>
kube-system	kube-proxy-4cphr	1/1	Running	0	99s	192.168.122.32	k8s-docker-worker	<none>	<none>
kube-system	kube-proxy-ggsgt	1/1	Running	0	2m31s	192.168.122.31	k8s-docker-master	<none>	<none>
kube-system	kube-scheduler-k8s-docker-master	1/1	Running	0	2m52s	192.168.122.31	k8s-docker-master	<none>	<none>

Figure 9: All the pods in the Kubernetes cluster and their status nodes with Docker runtime

CRI-O with Kubernetes

This is a lightweight CRI runtime, tailored specifically for Kubernetes high-level runtime. CRI-O works in coalition with any OCI runtime, to run pods, manage images and pull from any OCI-compatible image registry. We commonly see CRI-O running along with runC and Clear Containers as low-level runtimes.

The installation procedure is quite complex and involves many steps. The complete installation instructions are kept in our GitHub repository, and the link for the same can be found at <https://github.com/shubhamaggarwal890/nginx-vod/blob/master/kubernetes-CRI-o.md>.

Figure 6 shows all the nodes attached to the Kubernetes cluster, and we can see our installed master and worker nodes. The CONTAINER-RUNTIME column shows the installed container runtime along with its

version. Figure 7 showcases all the pods attached to the name spaces. We can see certain pods running on the master node and on the worker node under the NODE column.

Docker Engine with Kubernetes

Docker was originally developed as a monolithic daemon, but it has evolved over time. Its key components are now distributed, and it supports CRI through containerd. In current versions, containerd is installed along with Docker and interacts with CRI. It manages and runs images.

The installation procedure is quite complex and involves many steps. The complete installation instructions are kept in our GitHub repository, and the link for the same can be found at <https://github.com/shubhamaggarwal890/nginx-vod/blob/master/kubernetes-docker.md>.

Figure 8 shows all the nodes attached to the Kubernetes cluster, where we can see our installed master and worker nodes. The CONTAINER-RUNTIME column shows the installed container runtime along with its version. Figure 9 showcases all the pods attached to the name spaces. We can see certain pods running on the master node and on the worker node under the NODE column.

Containerd with Kubernetes

Containerd is one of the most popular CRI runtimes today. It is designed to fulfill container life cycle and image management, and is considered resource-efficient and more focused than Docker.

The installation procedure is quite complex and involves many steps. The complete installation instructions are kept in our GitHub repository, and the link for the same